ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA



ASSIGNMENT 1

Distributed and Streaming-based Client-Server Collaboration Application

Thành viên	Mã số sinh viên	Đóng góp
Phan Anh Khoa	2411647	100%
Phan Thanh Bình	2410357	100%
Nguyễn Đức Thái Huy	2411209	100%
Nguyễn Hữu Thắng	2413238	100%
Lê Quốc Thái	2313097	100%

Giảng viên hướng dẫn: Bùi Xuân Giang

Thành phố Hồ Chí Minh, tháng 11 năm 2026

Mục lục

1 Giới thiệu **4**

2 Kiến trúc hệ thống **5**

2.1 Sơ đồ tổng quan 5

2.2 Ba lớp và các module 6

3 Đặc tả giao thức **7**

3.1 Mặt phẳng điều khiển qua TCP 7

3.2 Header gói dữ liệu UDP 7

3.3 Phong bì relay `RLY1` 8

3.4 Stream và chống Head-of-Line blocking 9

3.5 Độ tin cậy: sequence number, ACK và SACK 9

3.6 Điều khiển tắc nghẽn CUBIC 9

3.7 Mã hóa payload 10

3.8 Quy trình truyền file 10

3.9 Sơ đồ trạng thái 10

4 Sequence Diagram **12**

4.1 Đăng nhập và đồng bộ danh sách online 12

4.2 Vòng đời một tin nhắn 12

4.3 Vòng đời truyền file 12

5 Challenges & Solutions **13**

6 Kiểm thử **14**

7 Kết luận, hạn chế và tài liệu tham khảo **17**

Chương 1

Giới thiệu

Ứng dụng dùng TCP cho đăng nhập và danh sách người online, còn chat và truyền file chạy trên giao thức tin cậy tự thiết kế trên UDP (cửa sổ trượt, SACK, CUBIC, AES-128-CBC), qua một Central Server và client có Web UI cục bộ.

Bài làm đáp ứng đủ 5 tính năng của đề: đăng nhập và khám phá, chat chung và riêng, truyền file tin cậy, chịu lỗi và mã hóa. Ứng dụng được triển khai bằng Python 3.9+, sử dụng socket thô, `threading` và `http.server` từ thư viện chuẩn. Hệ thống không dùng framework HTTP hay WebSocket; thư viện ngoài duy nhất là `cryptography`.

Mã nguồn GitHub: Kho mã nguồn trên GitHub

Tính năng	**Cách hiện thực**	**Module chính**
Đăng nhập, danh sách online	TCP bền vững, thông điệp NDJSON, `session_id` 32 bit và roster được đẩy tới mọi client.	`central_server.py`, `central_client.py`
Chat chung và riêng	UDP qua relay; stream 0 dành cho chat chung, mỗi cặp người dùng có stream riêng.	`socket_engine.py`
Truyền file tin cậy	Chia chunk 1350 byte, cửa sổ trượt, SACK, CUBIC và kiểm tra SHA-256 sau khi ghép file.	`reliability.py`, `streaming.py`
Chịu lỗi	Ping 3 giây, timeout 15 giây, tối đa 12 lần gửi lại một gói, `try/except` quanh mọi thao tác socket.	`central_server.py`, `reliability.py`
Mã hóa	AES-128-CBC, đệm PKCS7 và IV ngẫu nhiên cho từng gói.	`crypto.py`
Giao diện	Web UI thuần JavaScript tại `127.0.0.1:8080`; dashboard server tại `127.0.0.1:9003`.	Thư mục `web/`

Cách chạy nhanh (xem hướng dẫn chi tiết trong `README.md`):

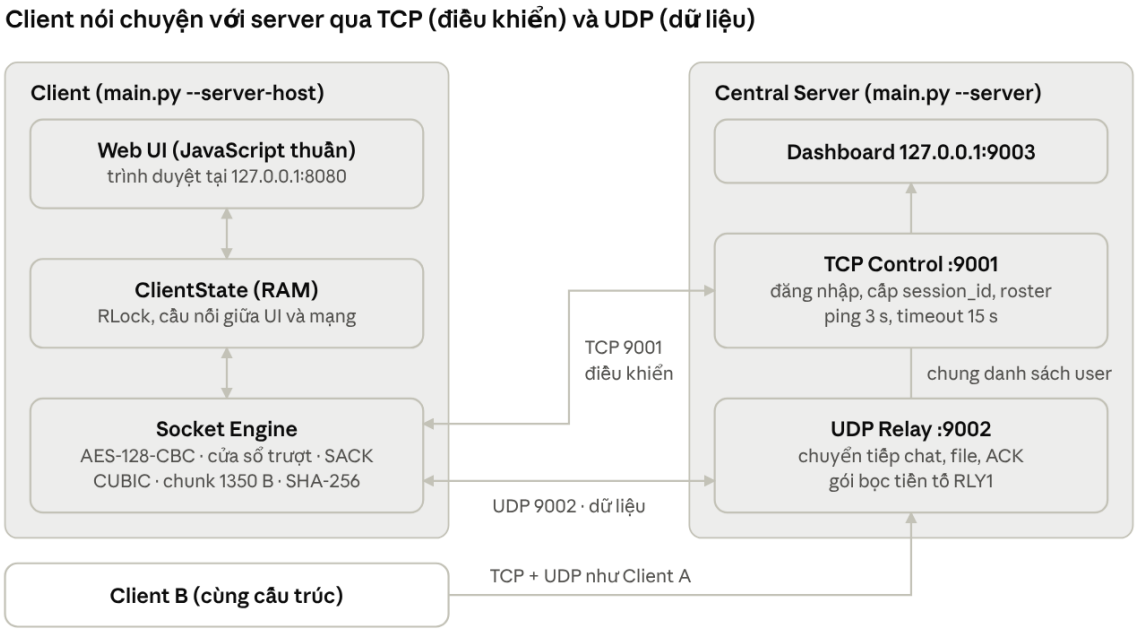
```
pip install -r requirements.txt
python main.py --server --host 0.0.0.0 --tcp-port 9001 --relay-port 9002
python main.py --server-host <IP_SERVER> --tcp-port 9001 --relay-port 9002 --udp-port 9010 --web-port 8080
```

Chương 2

Kiến trúc hệ thống

2.1 Sơ đồ tổng quan

Hệ thống theo mô hình lai: mỗi client kết nối tới một Central Server bằng hai kênh tách biệt, kênh điều khiển TCP (đăng nhập, danh sách online, ping) và kênh dữ liệu UDP (chat, file, ACK) đi qua relay của server. Bên trong client, ba lớp được tách rõ: Web UI, `ClientState` dùng chung trong RAM, và Socket Engine. Khi không đặt `--server-host`, client chạy chế độ P2P thuần với UDP broadcast discovery.



Client A và Client B cùng chạy một cấu trúc; chúng không gửi gói trực tiếp cho nhau mà đi qua UDP Relay của server.

2.2 Ba lớp và các module

Phía client được chia thành ba lớp chạy trong cùng tiến trình: Web UI, `ClientState` dùng chung trong RAM, và Socket Engine. UI đọc/cập nhật trạng thái qua `ClientState`; lớp trạng thái làm cầu nối an toàn đa luồng tới Socket Engine, nơi xử lý network và các gói tin.

Mạng

Module	Vai trò
`network/socket_engine.py`	Điều phối: nhận UDP, giải mã, gửi ACK/SACK, vòng lặp gửi lại mỗi 50 ms.
`network/reliability.py`	`SlidingWindowSender` (cửa sổ trượt, CUBIC, SACK) và `StreamReceiver` (sắp thứ tự, khử trùng lặp).
`network/central_server.py`	
`network/central_client.py`	TCP đăng nhập, ping, roster; server đồng thời relay UDP.

Xử lý dữ liệu

Module	Vai trò
`core/protocol.py`	`Packet`: header 24 byte, CRC, mã hóa và giải mã khối SACK.
`core/crypto.py`	`CryptoEngine` AES-128-CBC.
`network/streaming.py`	Cắt chunk 1350 byte, SHA-256, ghép file.
`core/state.py`	`ClientState` an toàn đa luồng (`RLock`), cầu nối giữa mạng và UI.

Giao diện

Module	Vai trò
`web/server.py`	
`web/static/js/app.js`	HTTP loopback và UI JavaScript thuần, polling mỗi 500 ms.
`web/central_dashboard.py`	Dashboard quản trị server: uptime, user, throughput, tiến độ file, kick.

Mô hình đồng thời. Dùng `threading`. Server có một thread accept TCP, một thread relay UDP và một thread cho mỗi client đang kết nối. Client có thread nhận UDP, thread gửi lại, thread đọc roster từ TCP, thread ping, thread HTTP; mỗi lần gửi file chạy trên một thread riêng.

Dữ liệu dùng chung được bảo vệ bằng `RLock`; server sao chép danh sách người dùng trong khóa rồi mới gửi ra ngoài khóa, nên một client chậm không giữ khóa của cả server. `tools/load_test.py` chạy 20 client đồng thời để kiểm tra mức 10–20 client của đề.

Chương 3

Đặc tả giao thức

Giao thức có hai tầng: mặt phẳng điều khiển dùng JSON trên TCP, mặt phẳng dữ liệu dùng gói nhị phân tự thiết kế trên UDP với header cố định 24 byte.

3.1 Mặt phẳng điều khiển qua TCP

Mỗi thông điệp là một dòng JSON kết thúc bằng ký tự xuống dòng. Client giới hạn mỗi dòng gửi tối đa 4096 byte. Bảng sau mô tả các thông điệp chính trên cổng TCP 9001.

Thông điệp	Hướng		Trường	Ý nghĩa
`login`	Client → Server	→	`username`, `udp_port`	Yêu cầu đăng nhập.
`login_ack`	Server → Client	→	`success`, `session_id`, `peers[]` hoặc `error`	Cấp phiên và trả danh sách peer. Server từ chối tên rỗng, tên đã online (không phân biệt hoa thường) hoặc cổng UDP ngoài khoảng 1–65535.
`roster`	Server → Client	→	`users[]`	Cập nhật danh sách online khi có người vào, rời đi hoặc bị kick.
`ping`	Client → Server	→	Không có	Gửi mỗi 3 giây; server loại client nếu im lặng quá 15 giây.
`warning`	Client → Server	→	`category`	Báo các bất thường như `invalid_packet`, `decrypt_failure` hoặc `file_decode_failure` lên dashboard.

3.2 Header gói dữ liệu UDP

Header cố định dài 24 byte, được đóng gói theo định dạng `!HBBIIIIHH` (big-endian, thứ tự byte mạng), sau đó là payload.

Offset	Trường	Kích thước	Mô tả
0	Magic	2 byte	Giá trị `0x4E41`; gói sai Magic bị loại.
2	Version	1 byte	Phiên bản hiện tại là 1.
3	Flags	1 byte	Bitmask loại gói, mô tả ở bảng kế tiếp.
4	Session ID	4 byte	ID phiên do server cấp cho người gửi.
8	Stream ID	4 byte	Định danh luồng logic của gói.
12	Seq Num	4 byte	Số thứ tự trong stream, bắt đầu từ 1.
16	ACK Num	4 byte	ACK tích lũy: số thứ tự tiếp theo bên nhận đang chờ.
20	Payload Len	2 byte	Độ dài payload tính theo byte.
22	Checksum	2 byte	16 bit thấp của CRC-32 trên header (checksum đặt bằng 0) và payload.

Bên nhận loại gói nếu thiếu byte, sai Magic, độ dài payload không khớp kích thước thực hoặc checksum không hợp lệ.

Flag	Giá trị	Công dụng
`FLAG_MSG`	`0x04`	Tin nhắn chat; payload được mã hóa.
`FLAG_FILE`	`0x08`	Metadata hoặc chunk file; payload được mã hóa.
`FLAG_ACK`	`0x10`	Xác nhận; payload chứa các khối SACK.
`FLAG_DISCOV`	`0x02`	Broadcast khám phá peer, chỉ dùng ở chế độ P2P.
`FLAG_HEARTBEAT`	`0x20`	Duy trì peer, chỉ dùng ở chế độ P2P.
`FLAG_AUTH`, `FLAG_SYN`, `FLAG_FIN`	`0x01`, `0x40`, `0x80`	Đã định nghĩa để mở rộng; luồng hiện tại đăng nhập qua TCP nên chưa sử dụng.

3.3 Phong bì relay `RLY1`

Trong chế độ có server, client mã hóa dạng Base64 (encoding, không phải mã hóa bảo mật) gói nhị phân rồi bọc trong datagram có tiền tố `RLY1`. Client gửi datagram tới UDP Relay; relay kiểm tra và chuyển tiếp tới client đích.

```
Client -> Server: RLY1{"type":"relay","sender_session_id":S,"target_session_id":T,"packet":"<base64>"}
Server -> Client: RLY1{"type":"deliver","sender_session_id":S,"packet":"<base64>"}
```

Giá trị `target_session_id = 0` biểu thị broadcast tới các client còn lại; client hiện gửi riêng một bản cho từng peer. Gói file bổ sung metadata `file` gồm `transfer_id`, `filename`, `filesize`, `total_chunks`, `first_seq`, cùng `retry_count` để dashboard theo dõi tiến độ và số lần gửi lại.

Trước khi chuyển tiếp, server xác minh IP và cổng gửi khớp phiên đăng nhập; Session ID trong gói khớp người gửi; tên file không chứa đường dẫn; kích thước file không quá 200 MB; số chunk phù hợp kích thước; sequence number nằm trong khoảng đã khai báo; và số lần gửi lại không vượt quá 12.

3.4 Stream và chống Head-of-Line blocking

Stream ID	Mục đích
`0`	Chat chung.
CRC-32 của chuỗi `"<id nhỏ>:<id lớn>"`	Chat riêng giữa hai người; cộng 2 nếu giá trị CRC không lớn hơn 1 để hai bên cùng tính ra ID hợp lệ.
Số ngẫu nhiên từ 2 đến $2^{32} - 1$	Mỗi lần gửi file.

Mỗi cặp (Stream ID, peer) có bộ gửi, bộ nhận, số thứ tự, cửa sổ trượt và `cwnd` riêng. Vì vậy, chunk file bị mất chỉ làm chậm stream file đó; tin nhắn trên stream khác vẫn được xử lý độc lập.

3.5 Độ tin cậy: sequence number, ACK và SACK

UDP không bảo đảm giao nhận, thứ tự hay chống trùng, vì vậy lớp tin cậy được xây dựng ở tầng ứng dụng. Mỗi gói trong một stream mang sequence number tăng dần từ 1, giúp bên nhận nhận biết gói nào đã đến, còn thiếu hoặc bị trùng.

Ở phía nhận, `StreamReceiver` chỉ nhận gói có seq trong $[expected, expected + 8)$; gói đến sớm được lưu vào buffer, gói trùng bị loại, và dữ liệu chỉ được giao lên tầng ứng dụng khi đủ thứ tự. Sau mỗi gói, bên nhận gửi ACK tích lũy (sequence number kế tiếp đang chờ) kèm các khối SACK. Payload SACK gồm Version (1 byte, giá trị 1), Block Count (1 byte, tối đa 32), rồi các khối là nửa khoảng $[\text{left}, \text{right})$, mỗi khối gồm hai số uint32 (8 byte), theo RFC 2018.

Ở phía gửi, `SlidingWindowSender` giữ số gói chưa ACK không quá $\min(8, cwnd)$; RTO cố định 0,5 giây; chỉ gửi lại gói chưa được SACK. Gói chưa xác nhận mà có ít nhất hai gói phía trên được SACK thì được gửi lại ngay (fast retransmit), không chờ RTO. Ngưỡng này đếm theo số gói được SACK, không theo số khối SACK. Nếu một gói bị gửi lại quá 12 lần, stream được đánh dấu thất bại và bên gửi báo lỗi thay vì chờ vô hạn.

Ví dụ. Giả sử cửa sổ đủ lớn để bên gửi phát liên tiếp gói 1 đến 5, còn bên nhận chỉ nhận được 2, 4 và 5. Bên nhận trả ACK với $ack = 1$ cùng hai khối SACK $[2, 3)$ và $[4, 6)$. Gói 1 có ba gói phía trên được SACK (2, 4, 5), còn gói 3 có hai gói (4 và 5); cả hai đạt ngưỡng nên được gửi lại ngay, dù gói 3 chỉ có một khối SACK phía trên. Khi gói 1 đến nơi, bên nhận giao liền gói 1 và 2; khi gói 3 đến nơi, bên nhận giao tiếp gói 3, 4 và 5, ACK tích lũy tiến lên 6.

3.6 Điều khiển tắc nghẽn CUBIC

`cwnd` khởi đầu ở 4 gói và tối đa 8 gói. Dưới `ssthresh`, slow start tăng một gói cho mỗi ACK. Sau đó, thuật toán dùng hàm CUBIC theo RFC 8312 với $C = 0{,}4$ và $\beta = 0{,}7$:

$$W(t) = C(t - K)^3 + W_{max}, \qquad K = \sqrt[3]{\frac{W_{max}(1 - \beta)}{C}}.$$

Đây là biến thể rút gọn dành cho UDP, không phải triển khai CUBIC đầy đủ của TCP: RTT trong công thức Reno được ước lượng từ RTO (không đo thực tế) và `cwnd` tối đa chỉ 8 gói.

Mỗi lần cập nhật, bên gửi so sánh giá trị CUBIC với mô hình TCP Reno thân thiện và chọn giá trị lớn hơn. Khi gói hết RTO, `cwnd` giảm một lần cho mỗi đợt mất gói: $W_{max} = cwnd$, $ssthresh = \max(2, 0{,}7\,cwnd)$, $cwnd = ssthresh$, rồi bắt đầu chu kỳ CUBIC mới.

3.7 Mã hóa payload

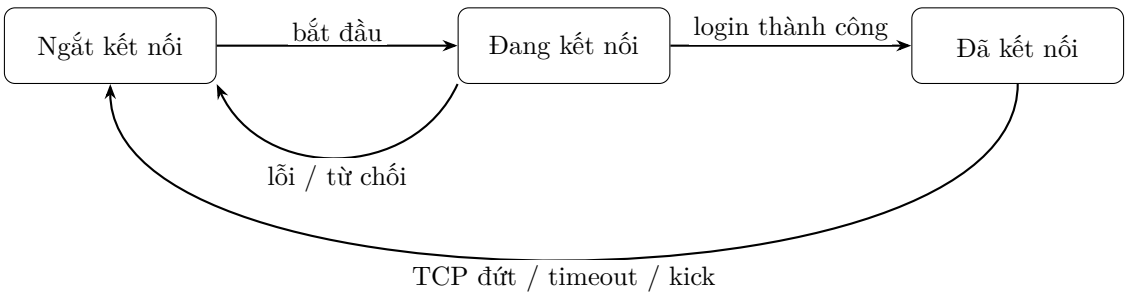
Payload của `FLAG_MSG` và `FLAG_FILE`, bao gồm metadata và từng chunk file, được mã hóa AES-128-CBC với đệm PKCS7. Mỗi gói có IV ngẫu nhiên 16 byte từ `os.urandom`, đặt ở đầu bản mã. Header và gói ACK/SACK không mã hóa. Khóa 16 byte lấy từ biến môi trường `CNET_SECRET_KEY`; nếu không đặt biến này, ứng dụng dùng khóa tĩnh trong mã nguồn. Các client phải dùng cùng khóa.

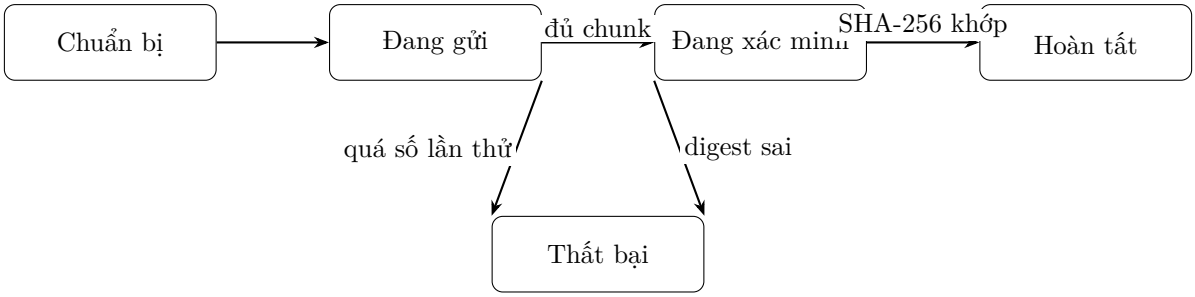
3.8 Quy trình truyền file

1. Bên gửi đọc file, chia thành chunk tối đa 1350 byte và tính SHA-256 toàn file.
2. Gói đầu stream file mang metadata JSON đã mã hóa: `filename`, `filesize`, `total_chunks`, `sha256`, `transfer_id` và `chat_stream_id`.
3. Các gói tiếp theo mang từng chunk đã mã hóa, đi qua cửa sổ trượt CUBIC riêng của stream file.
4. Bên nhận ghép đủ `total_chunks` theo thứ tự và kiểm tra SHA-256. Nếu khớp, file được lưu vào `storage/downloads/`; nếu trùng tên, thêm hậu tố `_1`, `_2`, v.v. Nếu digest lệch, báo "Nhận thất bại".

3.9 Sơ đồ trạng thái

Phiên client có ba trạng thái chính; truyền file có vòng trạng thái riêng bên trong phiên đã kết nối. Khi TCP đứt, quá 15 giây không nhận ping hoặc bị kick trên dashboard, client trở về trạng thái ngắt kết nối và xóa danh sách peer.




Chuẩn bị
Đang gửi
đủ chunk
Đang xác minh
SHA-256 khớp
Hoàn tất
quá số lần thử
digest sai
Thất bại

Chương 4

Sequence Diagram

Các sơ đồ mô tả luồng đăng nhập trên TCP và luồng dữ liệu tin nhắn, file qua UDP Relay. Thời gian đi từ trên xuống.

4.1 Đăng nhập và đồng bộ danh sách online

Client đăng nhập qua TCP. Server cấp `session_id`, trả danh sách peer hiện có và gửi `roster` cập nhật tới các client khác.

Xem sơ đồ đăng nhập và đồng bộ danh sách online

4.2 Vòng đời một tin nhắn

Ví dụ Client A gửi tin riêng cho Client B; gói chuyển tiếp lần đầu bị mất trên đường tới B nên không có ACK, và A gửi lại sau khi hết RTO. Với chat chung, client gửi một bản riêng cho từng peer; mỗi peer có cửa sổ trượt và ACK riêng nên một peer chậm không chặn cửa sổ hay ACK của peer khác. Tuy nhiên vòng gửi chat chung chạy tuần tự theo từng peer và có thể chờ tối đa 5 giây khi cửa sổ của một peer đầy, nên peer đó vẫn có thể làm trễ lượt gửi cho các peer đứng sau.

Xem sơ đồ vòng đời một tin nhắn

4.3 Vòng đời truyền file

File chạy trên một stream ngẫu nhiên riêng, dùng chung cơ chế ACK/SACK nhưng không chặn chat trên các stream khác.

Xem sơ đồ vòng đời truyền file

Chương 5

Challenges & Solutions

1. **Mất, trùng và đảo thứ tự gói UDP.** Khi mô phỏng mất 10–15% gói, chat có thể thiếu, trùng hoặc sai thứ tự, còn file có thể hỏng. Giao thức gắn sequence number riêng từng stream; receiver gửi ACK tích lũy và SACK, buffer gói đến sớm, loại gói trùng và giao lên đúng thứ tự. Sender chỉ truyền lại gói chưa được SACK, khi hết RTO 0,5 giây hoặc fast retransmit khi đã có từ hai gói phía trên được SACK. Kiểm chứng: `test_04_private_chat_in_order_exactly_once_with_loss` (20 tin, mất 15%) và `test_03_file_survives_10_percent_packet_loss`.

2. **Head-of-Line blocking giữa chat và file.** Nếu dùng chung chuỗi số thứ tự và cửa sổ, việc gửi lại một chunk file có thể chặn tin nhắn. Mỗi (Stream ID, peer) có bộ gửi, bộ nhận, cửa sổ trượt và `cwnd` riêng. Kiểm chứng: `test_05_chat_is_not_blocked_by_a_lossy_file_transfer` gửi 5 tin trong khi truyền file 400 KB ở mức mất gói 15%.

3. **Kích thước gói và phân mảnh IP.** Chunk 1350 byte, IV 16 byte, padding PKCS7 và header 24 byte giữ gói trực tiếp tối đa khoảng 1400 byte, dưới MTU 1500. Khi qua relay, Base64 và JSON làm datagram tăng lên khoảng 2 KB và có thể bị phân mảnh IP; SACK hỗ trợ gửi lại, còn hướng cải thiện là thay phong bì Base64 bằng định dạng nhị phân.

4. **Tranh chấp dữ liệu giữa các luồng.** Thread nhận UDP, gửi lại, HTTP và ping cùng truy cập trạng thái. `RLock` bảo vệ `ClientState` và bảng sender/receiver; mỗi socket TCP có `send_lock` riêng; UI chỉ đọc bản sao. Khi phát roster, server sao chép danh sách trong khóa rồi gửi ngoài khóa để client chậm không khóa toàn server.

5. **Client rớt đột ngột và peer ma.** Ping mỗi 3 giây, timeout TCP 15 giây giúp server loại client và phát roster mới. Sender bỏ cuộc sau 12 lần gửi lại một gói và báo lỗi; vòng đọc socket bắt `OSError`, lỗi JSON và `ValueError` để một client lỗi không làm sập server. Kiểm chứng: `test_11_silent_client_is_removed_after_timeout` và `test_06_sender_gives_up_when_receiver_disappears`.

Chương 6

Kiểm thử

Bộ kiểm thử gồm 101 test case, chạy với server thật và socket engine thật trên localhost. Mất gói giả lập được chèn ở socket UDP phía client (hàm gửi của client), không áp dụng cho chặng server đến client.

Kịch bản	File	Số test	Kiểm chứng
Bắt tay TCP, đăng nhập	`test/test_step2.py`	11	Nhận ACK, ghi IP và cổng UDP, từ chối tên rỗng, tên trùng, cổng sai, JSON hỏng; 10 client đăng nhập đồng thời; client im lặng bị loại sau timeout.
Xác thực và trạng thái người dùng	`test/test_step3.py`	10	Session ID duy nhất, thuộc uint32; cắt khoảng trắng tên; từ chối tên trùng mà không thay phiên cũ, tên rỗng/chỉ khoảng trắng và cổng UDP ngoài 1–65535; người mới nhận danh sách online; xóa phiên khi ngắt kết nối.
Kết nối, ngắt kết nối đồng thời	`test/test_step4.py`	10	10 client đăng nhập cùng lúc với session ID khác nhau; chỉ một phiên thành công khi đăng nhập trùng tên đồng thời; ngắt kết nối đồng thời, kết nối lại cùng tên, churn không để lại phiên cũ; metrics đúng.
Định tuyến và broadcast	`test/test_step5.py`	10	Tin riêng đúng người nhận, không lộ cho người khác; broadcast không echo người gửi; tiếng Việt/emoji qua mã hóa và relay; đúng thứ tự trong stream, các stream tách biệt; tên gửi do server xác thực; từ chối người nhận không tồn tại; nhiều người gửi tới một người nhận.
Truyền file UDP	`test/test_step6.py`	10	Chunk tối đa 1350 byte; metadata kích thước, số chunk, SHA-256; ghép đúng thứ tự; digest sai không lưu file; file rỗng; tên trùng không ghi đè (hậu tố `_1`); truyền qua relay file nhỏ, nhiều chunk, rỗng và nhị phân.
Timeout và chịu lỗi	`test/test_step7.py`	10	JSON TCP hỏng, yêu cầu đăng nhập thiếu, client im lặng bị loại sau timeout, TCP ngắt đột ngột, datagram UDP lạ, JSON relay hỏng, Base64 sai, gói nhị phân hỏng, relay từ người gửi chưa đăng ký; tắt server êm, server không sập.
Mã hóa	`test/test_step8.py`	10	Giải mã đúng văn bản tiếng Việt, dữ liệu nhị phân, payload lớn; IV ngẫu nhiên làm hai bản mã khác nhau; chuẩn hóa độ dài khóa.
Độ tin cậy và CUBIC	`test/test_retransmission.py`	12	Cửa sổ trượt, ACK tích lũy, SACK, chỉ gửi lại gói thiếu, RTO, giảm `cwnd` khi mất gói, bỏ cuộc sau 12 lần; sơ đồ tuần tự mất gói và SACK.
Đầu-cuối có mất gói	`test/test_e2e_network.py`	6	File 1 MB không mất gói; file 150 KB mất 10%; file qua relay mất 10%; chat riêng đúng thứ tự và đúng một lần khi mất 15%; chat không bị chặn bởi file; bên gửi bỏ cuộc khi bên nhận biến mất. Các file nhận được so SHA-256 với bản gốc.
Dashboard server	`test/test_central_dashboard.py`	3	Trạng thái, xuất và xóa log, dừng và khởi động lại server, kick client.
Tranh chấp tài nguyên	`test/test_thread_safety.py`	9	Nhiều luồng cùng đọc/ghi `ClientState` (outbox, `add_message`, snapshot) mà không mất hay trùng dữ liệu.
Tải 10–20 client	`tools/load_test.py`	–	20 client đăng nhập cùng lúc, mọi client nhận đủ tin chat chung của những client còn lại; tùy chọn gửi thêm file 1 MB.

Chạy từ thư mục gốc của dự án:

```
python -m test.run_tests
python tools/load_test.py --clients 20 --file-mb 1
```

Ngoài ra có kiểm thử thủ công theo `README.md`: một server và hai client trên localhost trong ba cửa sổ terminal, và nhiều máy trong cùng mạng LAN.

Kết quả chạy:

- Hình 6.1: Ảnh chụp terminal sau `python -m test.run_tests`, thấy dòng tổng kết.

- Hình 6.2: Ảnh chụp terminal sau `python tools/load_test.py -clients 20 -file-mb 1`, thấy `RESULT: PASS`.
- Hình 6.3: Ảnh giao diện web của hai client đang chat và gửi file, thấy danh sách online và thanh tiến độ.
- Hình 6.4: Ảnh dashboard server tại `127.0.0.1:9003`.

Chương 7

Kết luận, hạn chế và tài liệu tham khảo

Ứng dụng đáp ứng 10 bước của rubric: giao thức tin cậy tự thiết kế trên UDP, tách stream để giảm Head-of-Line blocking, điều khiển tắc nghẽn CUBIC, mã hóa payload và giao diện web JavaScript thuần.

Bước	Nội dung	Nơi hiện thực	Kiểm chứng
1	Thiết kế giao thức, sơ đồ trạng thái	Chương 3, `docs/Protocol_Spec.md`	–
2	Bắt tay TCP	`central_server.py`, `central_client.py`	`test_step2.py`
3	Đăng nhập, theo dõi user, IP, cổng	Session ID, roster, `get_users()`	`test_step2.py`, `test_step3.py`
4	Đồng thời	Thread cho mỗi client TCP, thread relay UDP	`test_step4.py`, `load_test.py` (20 client)
5	Định tuyến chat chung và riêng	UDP Relay, stream 0 và stream riêng	`test_step5.py`, `test_e2e_network.py`
6	Truyền file tin cậy	`streaming.py`, `reliability.py`	`test_step6.py`, `test_e2e_network.py`
7	Chịu lỗi	Ping, timeout 15 giây, giới hạn 12 lần gửi lại	`test_step7.py`, `test_step2.py`, `test_e2e_network.py`
8	Mã hóa	`crypto.py` (AES-128-CBC)	`test_step8.py`
9	Web UI JavaScript thuần	Tin nhắn, tiến độ file, danh sách online	Kiểm thử thủ công (xem Hình 6.3)
10	Kiểm thử và chất lượng mã	`test/`, `tools/load_test.py`	Chương 6

Hạn chế đã biết.

- Khóa AES dùng chung, không có trao đổi khóa; CBC không kèm MAC nên không chống sửa đổi có chủ đích.
- Phong bì relay dùng Base64 nên datagram tăng lên khoảng 2 KB.
- Bên gửi chưa được báo khi bên nhận ghép file thất bại; bên nhận hiển thị "Nhận thất bại".

- Stream ID chat riêng (CRC-32) và stream ID file (ngẫu nhiên) có thể va chạm với xác suất rất nhỏ.
- Web UI dùng `http.server` của thư viện chuẩn Python, không dùng framework HTTP.
- Đăng nhập chỉ bằng tên hiển thị, không có mật khẩu hay xác thực danh tính.
- Web UI cục bộ trả header `Access-Control-Allow-Origin: *` nên trang web bất kỳ mở trong trình duyệt của người dùng có thể gọi API tại `127.0.0.1`.
- Mất gói giả lập trong kiểm thử chỉ được chèn ở phía client, chưa mô phỏng mất gói ở chặng server đến client.

Tài liệu tham khảo.

- RFC 8312: CUBIC for Fast Long-Distance Networks, `https://datatracker.ietf.org/doc/html/rfc8312`.
- RFC 2018: TCP Selective Acknowledgment Options, `https://datatracker.ietf.org/doc/html/rfc2018`.
- Mã nguồn, `README.md` và `docs/Protocol_Spec.md` của dự án.